OAUTH CONSENT;

```
import React, { useEffect, useState } from "react";
import { appParams } from "@/lib/app-params";
import { Button } from "@/components/ui/button";
import { ShieldCheck, Loader2 } from "lucide-react";
import AuthLayout from "@/components/AuthLayout";

// App-side OAuth consent page for the app's MCP server. The platform redirects
// AI clients here (see base44/mcp/config.json `consent_path`) with an opaque
// `ctx` handle — the authorization request itself lives on the server. This page
// gates on the app-user session, fetches the display info for that handle, shows
// the categories of access being granted, and posts the approve/deny decision.
// Do not change the fetch calls, headers, or the `ctx` handle handling — styling
// and copy are safe to edit.
export default function OAuthConsent() {
 const ctx = new URLSearchParams(window.location.search).get("ctx");
 const [info, setInfo] = useState(null);
 const [checking, setChecking] = useState(true);
 const [submitting, setSubmitting] = useState(false);
 const [decided, setDecided] = useState("");
 const [error, setError] = useState("");
 const [reconnect, setReconnect] = useState("");

 useEffect(() => {
  (async () => {
   let redirecting = false;
   try {
    if (!ctx) {
     setError("This authorization link is invalid or has expired.");
     return;
    }
    // Resolve the handle first: a dead handle must never render
    // approve/deny, and the response carries the app's configured login
    // route for the signed-out redirect below. Send the session (cookie +
    // bearer token) so the server can list the granted tools for a
    // signed-in user — the same auth the approve/deny call sends; without
    // it the display request is anonymous and shows no tools.
    const infoHeaders = {};
    if (appParams.token) infoHeaders.Authorization = "Bearer " + appParams.token;
    const res = await fetch(
     `/api/apps/${appParams.appId}/mcp/consent-info?handle=${encodeURIComponent(ctx)}`,
     { credentials: "include", headers: infoHeaders },
    );
    if (!res.ok) {
     setError("This authorization link is invalid or has expired.");
     return;
    }
    const data = await res.json();
    // Gate on the server's auth result, NOT base44.auth.isAuthenticated():
    // the SDK check runs the bearer path, so a cookie-only session (platform
    // login/SSO, or a private app with a stale localStorage token) would read
```

```
    // as signed-out and redirect — even though /consent-info just
    // authenticated this same request via its cookie fallback. data.authenticated
    // keeps the redirect decision in agreement with what the server returned.
    if (!data.authenticated) {
     // The short handle rides back in returnTo; login_path is
     // owner-configured and validated server-side as a same-origin path.
     // Send from_url too: a custom-auth app coerced to platform auth (e.g.
     // public_without_login under workspace SSO) serves the platform login,
     // which honors from_url rather than returnTo. Rebuild the query from
     // `ctx` alone — never forward window.location.search raw: the platform
     // resume returns from_url verbatim, so crafted extras on the consent
     // link (app_base_url, access_token, …) would ride through the login
     // round-trip and app-params.js would persist them into the freshly
     // authenticated session.
     const returnTo =
      window.location.pathname + "?ctx=" + encodeURIComponent(ctx);
     const encoded = encodeURIComponent(returnTo);
     redirecting = true; // keep the spinner while the browser navigates
     window.location.href =
      (data.login_path || "/login") + "?returnTo=" + encoded + "&from_url=" + encoded;
     return;
    }
    setInfo(data);
   } catch (e) {
    setError("Could not load this authorization request. Please try again.");
   } finally {
    if (!redirecting) setChecking(false);
   }
  })();
}, [ctx]);

const respond = async (action) => {
 setSubmitting(true);
 setError("");
 try {
  const headers = { "Content-Type": "application/json" };
  // Cookie-backed sessions carry no token; sending "Bearer null" would
  // shadow the valid cookie, so add the header only when a token exists.
  if (appParams.token) headers.Authorization = "Bearer " + appParams.token;
  const res = await fetch(`/api/apps/${appParams.appId}/mcp/authorize-grant`, {
   method: "POST",
   credentials: "include",
   headers,
   body: JSON.stringify({ ctx, action }),
  });
  if (!res.ok) {
   // 401 = the session expired before the (single-use, still-unconsumed)
   // handle was spent; retrying the same controls re-sends the dead session
   // forever. Send the user back through login preserving `ctx` — the same
   // redirect the initial signed-out path uses — so they can return and
   // approve the still-valid handle.
   if (res.status === 401) {
```

```
      const returnTo = window.location.pathname + "?ctx=" + encodeURIComponent(ctx);
      const encoded = encodeURIComponent(returnTo);
      window.location.href =
       ((info && info.login_path) || "/login") + "?returnTo=" + encoded + "&from_url=" +
encoded;
      return;
     }
     // These all come AFTER the single-use handle is atomically consumed
     // (409 tool set changed; 403 host/resource/app mismatch; 404 access
     // gone; 400 malformed/handle already used), so retrying can only 404.
     // Show a terminal reconnect state, not an impossible "try again".
     if ([400, 403, 404, 409].includes(res.status)) {
      let detail = "";
      try { detail = (await res.json()).detail; } catch (_) { /* keep default */ }
      setReconnect(detail || "This authorization can no longer be completed. Reconnect
from your AI client to try again.");
      setSubmitting(false);
      return;
     }
     throw new Error("Could not complete authorization. Please try again.");
    }
    const data = await res.json();
    window.location.href = data.redirect_url;
    if (!/^https?:/i.test(data.redirect_url)) {
     // Custom-scheme redirect (native AI clients, e.g. cursor://): browsers
     // may block or not visibly navigate, so show a terminal state instead
     // of an eternal spinner.
     setDecided(action);
     setSubmitting(false);
    }
   } catch (e) {
    setError(e.message);
    setSubmitting(false);
   }
  };

  if (checking) {
   return (
    <AuthLayout icon={ShieldCheck} title="Authorize access">
     <div className="flex items-center justify-center py-6 text-muted-foreground">
      <Loader2 className="w-5 h-5 mr-2 animate-spin" aria-hidden="true" />
      Loading…
     </div>
    </AuthLayout>
   );
  }

  const client = (info && info.client_name) || "An AI client";
  const appName = (info && info.app_name) || "this app";

  if (decided) {
   return (
```

```
    <AuthLayout
     icon={ShieldCheck}
     title={decided === "approve" ? "Access granted" : "Access denied"}
     subtitle={`You can return to ${client} and close this window.`}
    />
   );
  }

  // Terminal: the authorization request is no longer valid (tool set changed +
  // handle consumed). Retrying can't succeed, so show reconnect guidance with
  // no approve/deny controls.
  if (reconnect) {
   return (
    <AuthLayout icon={ShieldCheck} title="Reconnect required">
     <div className="p-3 rounded-lg bg-destructive/10 text-destructive text-sm">
      {reconnect}
     </div>
    </AuthLayout>
   );
  }

  // No consent details means nothing trustworthy to approve: a failed
  // consent-info load (expired handle, rate limit, transient error) renders
  // the error alone, never the approve/deny controls.
  if (error && !info) {
   return (
    <AuthLayout icon={ShieldCheck} title="Authorize access">
     <div className="p-3 rounded-lg bg-destructive/10 text-destructive text-sm">
      {error}
     </div>
    </AuthLayout>
   );
  }

  const tools = Array.isArray(info.tools) ? info.tools : [];

  return (
   <AuthLayout
    icon={ShieldCheck}
    title="Authorize access"
    subtitle={`${client} wants to access ${appName} on your behalf`}
   >
    {error && (
     <div className="mb-4 p-3 rounded-lg bg-destructive/10 text-destructive text-sm">
      {error}
     </div>
    )}

    <p className="text-sm font-medium text-foreground mb-2">
     {tools.length ? `It will be able to use these tools in ${appName}:` : "No tools
requested"}
    </p>
```

```
    {tools.length > 0 && (
     <ul className="space-y-2 text-sm mb-6">
      {tools.map((tool) => (
       <li key={tool.name} className="flex flex-col">
        <span className="text-foreground font-medium">
         {tool.title || tool.name}
        </span>
        {tool.description && (
         <span className="text-muted-foreground">{tool.description}</span>
        )}
       </li>
      ))}
     </ul>
    )}

    <div className="flex gap-3">
     <Button
      variant="outline"
      className="flex-1 h-12 font-medium"
      disabled={submitting}
      onClick={() => respond("deny")}
     >
      Deny
     </Button>
     <Button
      className="flex-1 h-12 font-medium"
      disabled={submitting}
      onClick={() => respond("approve")}
     >
      {submitting ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : null}
      Approve
     </Button>
    </div>
   </AuthLayout>
  );
}
```